QRStream Technical Engineering Reference

Purpose: A concise technical reference describing the implementation, module interactions, protocol, runtime flow, algorithms and design decisions of the QRStream offline file transfer system.

1. Overall Architecture

QRStream is divided into two executables:

Python Sender

- - Reads any input file.
- - Splits the file into fixed-size chunks.
- - Builds protocol packets.
- - Serializes packets into a binary format.
- - Base64 encodes the packet.
- - Generates a QR image using Segno.
- - Displays QR frames using OpenCV.

Android Receiver

- - Captures camera frames using CameraX.
- - Detects QR codes using ML Kit.
- - Decodes Base64 into binary packets.
- - Parses the custom QRStream packet.
- - Verifies CRC32.
- - Stores unique chunks.
- - Reconstructs the original file.
- - Saves it into Downloads/QRStream.

Pipeline:

File -> Chunks -> Packet -> Base64 -> QR -> Camera -> Decode -> Parse -> Verify -> Reconstruct -> Save

2. QRStream Protocol

Packet Header:

MAGIC (4B), VERSION (1B), TYPE (1B), TRANSFER UUID (16B),

CHUNK NUMBER (4B), TOTAL CHUNKS (4B), PAYLOAD SIZE (2B),

CRC32 (4B), PAYLOAD (N).

Packet Types:

START: JSON metadata (filename, total chunks, size).

DATA: File bytes.

END: Signals transmission completion.

Byte order: Big Endian.

CRC32 protects payload integrity.

UUID identifies a transfer session.

3. Sender Implementation

app.py orchestrates the transfer.

file_reader.py loads the source file.

file_splitter.py creates fixed-size chunks.

packet_builder.py creates START/DATA/END packets.

serializer.py converts packet objects into binary.

qr_generator.py converts Base64 text into QR images.

qr_display.py renders QR frames and progress.

scheduler.py determines transmission order.

transmission_engine.py controls display timing.

config.py contains chunk size, FPS, repeats and QR settings.

4. Android Receiver

MainActivity launches the Compose UI.

CameraScreen coordinates the transfer state.

CameraPreview binds CameraX Preview and ImageAnalysis.

CameraAnalyzer scans QR codes using ML Kit and filters duplicates.

PacketParser reconstructs Packet objects from binary.

ReceiverCore validates CRC32, tracks START/DATA/END packets,

stores unique chunks in a TreeMap and detects missing chunks.

FileWriter merges ordered chunks and saves the reconstructed file.

StatusOverlay shows status and progress.

5. Runtime Execution

1. Sender reads the file.

2. File is split into chunks.

3. START packet is sent repeatedly.

4. DATA packets are displayed as QR frames.

5. Android camera captures each QR.

6. ML Kit decodes Base64.

7. PacketParser extracts header and payload.

8. ReceiverCore validates CRC and stores chunks.

9. END packet arrives.

10. Missing chunk check.

11. Ordered reconstruction.

12. FileWriter stores the file in Downloads/QRStream.

6. Algorithms

File Splitting: Sequential fixed-size chunking.

Packet Construction: Header + Payload + CRC.

Duplicate Removal: Ignore previously stored chunk numbers.

Reconstruction: TreeMap ensures chunk ordering before writing.

CRC Validation: Compute CRC32(payload) and compare to packet header.

7. Design Decisions

TreeMap: Maintains sorted chunk order.

Base64: Ensures binary can be embedded in QR text.

Segno: High-quality QR generation.

CameraX: Stable camera pipeline.

ML Kit: Fast QR decoding.

MediaStore: Compatible file saving on Android 10+.

Big Endian: Consistent cross-platform serialization.

Duplicate filtering: Prevents repeated scans from corrupting progress.

8. Performance & Limitations

Performance depends on QR size, display FPS, lighting, focus and camera quality.

Lower FPS increases reliability.

START/END repetition improves synchronization.

Missing chunks are detected before reconstruction.

Large files require more QR frames, increasing total transfer time.

Recommended tuning:

- - Chunk size around 700 bytes.
- - Display FPS 5–8 for reliable decoding.
- - Adequate lighting and stable camera.